

Betriebsdokumentation

Installation, Konfiguration und Betrieb (lokal und auf Railway)

PROJEKT

Lernzeit-Manager
ISEF01 · Projekt Software Engineering

STAND

21. August 2026

TEAM

Elias Ebertshäuser • Assis Ramadan • Julian Wagner

1. Überblick

Dieses Dokument beschreibt, wie der Lernzeit-Manager installiert, konfiguriert und betrieben wird — sowohl lokal für die Entwicklung als auch in der Produktionsumgebung auf Railway.

2. Systemvoraussetzungen

■ 2.1 Lokale Entwicklung

KOMPONENTE	VERSION	VERWENDUNG
Python	3.12	Backend-Laufzeit
Node.js	22 LTS	Frontend-Build und Dev-Server
npm	10.x	Paketmanager für Frontend
Docker Desktop	Aktuell	Lokale PostgreSQL-Datenbank
Git	Aktuell	Versionskontrolle

Empfohlene IDE: VS Code mit den Extensions **Python**, **Pylance**, **Ruff**, **Angular Language Service**, **ESLint** und **Docker**.

■ 2.2 Produktion (Railway)

Railway stellt automatisch alle Laufzeitumgebungen bereit. Kein manuelles Setup nötig. Ein Railway-Account und Zugriff auf das GitHub-Repository sind ausreichend.

3. Lokale Installation

3.1 Repository klonen

```
git clone https://github.com/Cooke0/Projekt-Lernzeit_Manager.git
cd Projekt-Lernzeit_Manager
```

3.2 Lokale Datenbank starten

Die `docker-compose.yml` liest ihre Werte aus einer `.env`-Datei im Projekt-Wurzelverzeichnis. Erstelle diese Datei einmalig mit folgendem Inhalt:

```
POSTGRES_USER=lernzeit
POSTGRES_PASSWORD=lernzeit_dev
POSTGRES_DB=lernzeit
POSTGRES_PORT=5432
```

Danach Datenbank starten:

```
docker compose up -d
```

PostgreSQL ist jetzt auf `localhost:5432` erreichbar. Die Standard-`DATABASE_URL` im Backend (`postgresql://lernzeit:lernzeit_dev@localhost:5432/lernzeit`) passt zu diesen Werten.

EINSTELLUNG	WERT
Host	localhost
Port	5432
Datenbankname	lernzeit
Nutzer	lernzeit
Passwort	lernzeit_dev

3.3 Backend einrichten

```
cd backend
python -m venv .venv

# Windows:
.venv\Scripts\activate
# macOS/Linux:
source .venv/bin/activate

pip install -r requirements-dev.txt
```

Datenbankmigrationen einmalig einspielen:

```
flask db upgrade
```

Backend starten:

```
flask run  
# Backend läuft auf http://localhost:5000
```

Health-Check:

```
curl http://localhost:5000/api/health  
# → {"status": "ok"}
```

■ 3.4 Frontend einrichten

```
cd frontend  
npm ci  
ng serve  
# Frontend läuft auf http://localhost:4200
```

Der Angular-Dev-Server leitet alle `/api`-Anfragen automatisch an `http://localhost:5000` weiter (konfiguriert in `proxy.conf.json`).

■ 3.5 Playwright E2E-Tests einrichten (optional)

```
cd frontend  
npm ci  
npx playwright install chromium
```

Tests ausführen (beide Server müssen laufen):

```
npx playwright test
```

Testbericht öffnen:

```
npx playwright show-report
```

4. Umgebungsvariablen

4.1 Backend-Umgebungsvariablen

VARIABLE	BESCHREIBUNG	BEISPIELWERT	PFLICHT
<code>DATABASE_URL</code>	PostgreSQL-Verbindungs-URL	<code>postgresql://user:pw@host:5432/db</code>	Ja (Produktion)
<code>JWT_SECRET_KEY</code>	Geheimschlüssel für JWT-Token-Signierung	Zufälliger String ≥ 32 Zeichen	Ja (Produktion)
<code>SECRET_KEY</code>	Flask-Secret-Key, signiert die Anmelde-Ausweise	Zufälliger String	Ja (Produktion)
<code>FLASK_ENV</code>	Umgebungsmodus	<code>production</code>	Ja (Produktion)
<code>CORS_ORIGINS</code>	Erlaubte Ursprünge für Cross-Origin-Requests	<code>https://xyz.railway.app</code>	Nein

Sicherheitshinweis: Diese Variablen dürfen niemals in das Repository committet werden. Für die lokale Entwicklung können sie in einer `.env`-Datei im `backend/`-Ordner gespeichert werden — diese Datei ist in `.gitignore` eingetragen.

Die Variable `SECRET_KEY` muss im Railway-Dienst mit einem eigenen, zufälligen Wert gesetzt sein; sie signiert die Anmelde-Ausweise. Seit Plan P3 verweigert die Anwendung in der Produktivumgebung den Start, wenn die Variable fehlt, statt still auf einen im Quelltext stehenden Vorgabewert zurückzufallen. Ein neuer Wert lässt sich mit `python -c "import secrets; print(secrets.token_urlsafe(48))"` erzeugen. Wird der Wert gewechselt, werden alle bestehenden Anmeldungen ungültig und alle Nutzenden müssen sich neu anmelden.

Beispiel `backend/.env` (nur für lokale Entwicklung):

```
DATABASE_URL=postgresql://lernzeit:lernzeit_dev@localhost:5432/lernzeit
JWT_SECRET_KEY=lokaler-entwicklungs-schluesel-mindestens-32-zeichen
FLASK_ENV=development
```

4.2 Railway-Umgebungsvariablen setzen

Im Railway-Dashboard unter dem Projekt → **Variables**:

```
DATABASE_URL    → Wird automatisch gesetzt, wenn das PostgreSQL-Add-on hinzugefügt wird
JWT_SECRET_KEY  → Langen, zufälligen String eingeben (z. B. mit `openssl rand -hex 32` erzeugen)
FLASK_ENV       → production
CORS_ORIGINS    → https://projekt-lernzeitmanager-production-0412.up.railway.app
```

5. Railway-Deployment

Railway baut die Anwendung als **einen einzigen Dienst** (Frontend und Backend zusammen, siehe `MS4_Technische_Dokumentation.md` Abschnitt 5.2) über ein `Dockerfile` im Repository-Root; der Builder ist in `railway.json` fest auf `"DOCKERFILE"` gepinnt, damit Railway die Build-Methode nicht selbst erraten muss. Bis 2026-08-20 baute das Projekt stattdessen über Nixpacks; dieser Ansatz wurde verworfen, weil Railway den Nixpacks-Builder inzwischen als veraltet einstuft (Plan P13).

■ 5.1 Erstmaliges Deployment (bereits erledigt, Julian)

1. Railway-Account erstellen: <https://railway.app> (kostenloser Tier ausreichend) und mit dem GitHub-Account anmelden.
2. **New Project** → **Deploy from GitHub repo** → dieses Repository auswählen. Das Root-Verzeichnis bleibt der Repository-Root (nicht `backend/` oder `frontend/`), da das `Dockerfile` dort liegt.
3. **New** → **Database** → **Add PostgreSQL** hinzufügen. Railway setzt `DATABASE_URL` für den eigenen Dienst nicht automatisch — dafür im Backend-Dienst unter **Variables** `DATABASE_URL=${{Postgres.DATABASE_URL}}` referenzieren.
4. Übrige Umgebungsvariablen setzen (siehe Abschnitt 4.2).
5. Unter **Settings** → **Networking** → **Generate Domain** die öffentliche URL erzeugen.

■ 5.2 Folgende Deployments

Deployment passiert automatisch bei jedem Push auf `main` (GitHub-Integration, kein manuelles `railway up` nötig): Railway baut das `Dockerfile` neu und startet `start.sh`, das zuerst `flask db upgrade` und danach Gunicorn ausführt. Geht ein Deploy schief: `railway logs` lesen, Fix auf einem Branch, PR, Merge — kein Hotfix direkt auf `main`.

■ 5.3 Alltag mit der Railway-CLI

Die CLI ist für Deployment selbst nicht nötig (das übernimmt die GitHub-Integration), aber nützlich für Logs und lokale Befehle mit Produktions-Umgebungsvariablen:

```
railway login
railway link      # Repo einmalig mit dem Railway-Projekt verknüpfen
railway logs      # Live-Logs des Dienstes
railway run <befehl> # Befehl lokal mit Railway-Env-Variablen ausführen
```

■ 5.4 Datenbankzugriff

Für lokale DB-Inspektion der Produktionsdatenbank:

```
railway connect postgresql
```

6. Test-Accounts für den Tutor

Die folgenden Test-Accounts wurden für das Tutor-Review erstellt. Sie enthalten ausschließlich fiktive Daten ohne Bezug zu realen Personen.

Primärer Test-Account (mit Beispieldaten):

FELD	WERT
URL	https://projekt-lernzeitmanager-production-0412.up.railway.app
E-Mail	tutor@test.lernzeit.de
Passwort	Tutor2026!
Name	Tutor Testaccount

Zweiter Test-Account (leer, für eigene Tests):

FELD	WERT
E-Mail	demo@test.lernzeit.de
Passwort	Demo2026!
Name	Demo Account

Hinweis: Beide Accounts sind eingerichtet. Im primären Account (tutor@test.lernzeit.de) sind Beispiel-Lernziele, Planungseinträge und Lernsessions vorhanden, damit der Tutor alle Funktionen direkt im Browser ausprobieren kann.

Korrektur 2026-08-21: Die zuvor hier dokumentierte URL (`projekt-lernzeitmanager-production.up.railway.app` , ohne `-0412`) antwortet inzwischen mit HTTP 404 — sie ist nicht mehr die aktuelle Produktions-URL. Die oben stehende URL mit `-0412` wurde am 2026-08-21 als tatsächlich erreichbar verifiziert (`GET /api/health` liefert `{"status":"ok"}` , `GET /api/stats` liefert HTTP 401 statt 404 und beweist damit, dass der Dienst mindestens den Stand von Plan P11 ausliefert). **Nicht verifiziert** wurde in dieser Sitzung, ob die oben gelisteten Zugangsdaten auf dieser URL tatsächlich funktionieren — das hängt davon ab, ob es sich um dieselbe Datenbank/denselben Railway-Dienst handelt oder ob dieser beim Wechsel von Nixpacks auf den Dockerfile-Builder (Plan P13) neu angelegt wurde. Vor der Redmine-Abgabe unbedingt manuell mit den Zugangsdaten einloggen und prüfen, ob die Beispieldaten noch vorhanden sind; ggf. die Accounts neu anlegen.

Datenschutz: Die Passwörter werden serverseitig gehasht gespeichert. Im Repository sind sie in diesem Dokument als Plaintext nur für das Tutor-Review aufgeführt, da es sich um reine Test-Accounts ohne echte personenbezogene Daten handelt.

7. Backup und Wiederherstellung

Die PostgreSQL-Datenbank auf Railway wird im kostenlosen Tier ohne automatische Backups betrieben. Für Produktionsdaten wird empfohlen, regelmäßig einen manuellen Dump zu erstellen:

```
railway connect postgresql  
pg_dump $DATABASE_URL > backup_$(date +%Y%m%d).sql
```

Für Studienzwecke (MS4-Abgabe) ist ein Backup nicht zwingend erforderlich; bei Datenverlust können Test-Accounts neu angelegt werden.

8. Bekannte Einschränkungen

- Der kostenlose Railway-Tier setzt den Service nach 30 Minuten Inaktivität in Schlaf. Der erste Request nach einer Pause kann daher 5–10 Sekunden dauern (Cold Start).
- Kein automatisches Backup der Produktionsdatenbank im Free-Tier.
- Passwort-Zurücksetzen-Funktion nicht implementiert (nur für die Abgabe relevant).